High Throughput Large Integer Multipliers Built on Optimized FPGA DSP Block Primitives

***Abstract*—Large integer multiplication is critical in cryptographic applications such as Fully Homomorphic Encryption (FHE) and RSA where it represents the main step in computing the modular multiplication used to generate keys. Most previous work building such multipliers on FPGAs has overlooked the importance of considering the underlying architecture of the Digital Signal Processing (DSP) Blocks provided in modern FPGAs. These DSP blocks can operate at high frequency but present an irregular input size that must be tiled to create large multipliers. In this paper, we show how a near-optimial base multiplier can be built using modern DSP blocks, then scaled to large sizes, while maintaining resource efficiency and high throughput. We demonstrate the automation of tiling, and establish a new performance standard for large integer multipliers on FPGAs.**

I. Introduction

Large integer multiplication is widely used in various applications such as digital signal processing, image processing and cryptographic applications. However, these applications often require large wordlengths that exceed the capabilities of hardware resources Field-Programmable Gate Arrays (FPGAs) offer a flexible architecture that can be used to implement a variety of applications in domains such as image processing [1]–[3], digital signal processing [4]–[6], and cryptography [7]–[12].

Since multiplications are so prevalent across different applications, and would consume large numbers of the LUTs used for mapping general functions on FPGAs, DSP blocks have gained significant importance in providing high computational throughput, and have evolved to become more functional and flexible, offering high throughput and efficiency and lower silicon area than what can be achieved with LUTs. However, making the best use of both DSP blocks and LUTs for complex arithmetic implementations remains challenging, as mapping tools do not produce optimal mappings.

In order to multiply large numbers using smaller multipliers, it is necessary to decompose them into smaller pieces that can be mapped to the wordlengths of the underlying architecture. The Schoolbook decomposition is a simple scheme that uses shift and add operations. Comba multiplication [13] reorders partial products for further optimization. Further efficiency can be gained with Karatsuba decomposition [14], which reuses some partial products to reduce the number of multipliers needed, as illustrated in Section II. It is also possible to use the Fast-Fourier Transform (FFT) for extremely large wordlengths [15]–[17]. While traditional FFTs operate on complex numbers, the Number-Theoretic Transform (NTT) is sometimes preferred when performing large integer multiplication and is a special case of the FFT over a finite field Z [18], [19].

Decomposing large multipliers spatially across smaller primitives requires careful consideration. Many decomposition schemes have been proposed and explored mostly for symmetric power-of-2 wordlengths, while the primitives on FPGAs are both asymmetric and non-power-of-2. Furthermore, there are considerations relating to the mix of signed and unsigned parts and how they are mapped to the underlying architecture. DSP blocks also have specific pipelining arrangements that must be respected to achieve high performance. Additionally, the FPGA architecture offers additional possible routes for optimization including the use of the cascade paths between multipliers, that would further enhance mapping. This paper attempts to address this mapping in an architecture-centric manner, enabling large integer multipliers to be efficiently mapped to FPGAs.

The main contributions of this paper are:

1) A highly efficient design of a 78×78 base multiplier using DSP primitives, which operates at the maximum frequency of the device and uses the minimum number of DSP slices, offering an efficient basic unit of arithmetic.
2) A tool that automatically builds large integer multipliers from the proposed base multiplier from a high-level JSON description.
3) A comparative analysis of our base multiplier with an existing vendor 64×64 IP multiplier and evaluated performance for large multipliers built using both bases, as well as against the approaches presented in [20].

Our IP-based multiplier achieved around 45% higher frequency than other approaches while using resources inefficiently. Additionally, our Primitive-based approach achieved around 73% higher frequency than other approaches while using around 13% fewer DSP blocks at the expense of 5% more LUTs.

II. Background

A. Decomposition Schemes

In order to perform large integer multiplications, inputs must be decomposed into smaller multiplications, which can be achieved through various decomposition schemes. For instance, Schoolbook decomposition is a scheme that uses four $(n/2 \times n/2)$ multipliers to perform an $(n \times n)$ multiplication as in Equation 1: We denote the two multiplier inputs A, B of size n, where A_H denotes the upper $n/2$ bits of A, and A_L denotes the lower $n/2$ bits of A, and similarly for B.

$$\begin{aligned} A &= (A_H << n/2) + A_L \\ B &= (B_H << n/2) + B_L \\ Schoolbook(A \times B) &= (A_L B_L) \\ &\quad + (A_L B_H + A_H B_L) << n/2 \\ &\quad + (A_H B_H) << n \end{aligned} \tag{1}$$

Equation 1 can be re-ordered to save an addition as follows in Equation 2 [20].

$$\begin{aligned} Schoolbook(A \times B) &= (A_L B_L \ \& \ A_H B_H) \\ &\quad + (A_L B_H + A_H B_L) << n/2 \end{aligned} \tag{2}$$

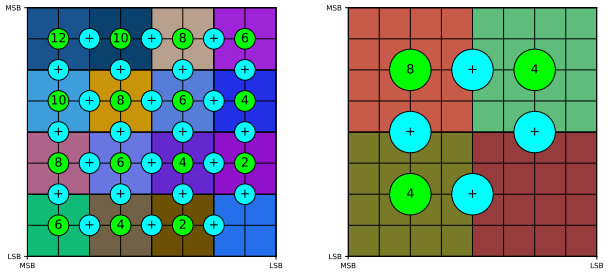
The Schoolbook decomposition method suffers from quadratic scaling complexity, requiring nine multipliers when inputs are divided into three parts. These partial products need additional adder and shifter circuits for final result computation.

The Karatsuba algorithm [14] improves upon the Schoolbook method by reducing the required multipliers from four to three, trading multiplications for more additions and subtractions, a favorable trade-off as they are cheaper than multiplication. In addition, one of the three multipliers is slightly larger, that is $(n/2 + 1)$ bits. Equation 3 illustrates how Karatsuba substitutes two smaller multipliers with one slightly larger multiplier and additional arithmetic operations.

$$\begin{aligned} Karatsuba(A_L B_H + A_H B_L) &= (A_L + A_H)(B_L + B_H) \\ &\quad - A_L B_L \\ &\quad - A_H B_H \end{aligned} \tag{3}$$

B. Tiling Schemes

Having understood how to decompose multiplications, we can now discuss how to tile smaller multipliers spatially to build large ones. Tiling problem is NP-complete [21]–[23], but an optimal solution can be found for small multipliers, such as 64×64 bits [24] Thus, existing research has explored algorithms to solve the general problem of tiling as in [25]–[29]. Every bit of the larger rectangle must be covered exactly once, that is, tiles can not overlap in a valid solution. This introduces a tradeoff between using many small tiles that offer greater flexibility in decomposition but require extensive shift-and-add operations to combine partial results, or fewer large tiles that minimize arithmetic operations but may lead to inefficient hardware mapping.



(a) Tiling 8×8 using 2×2. (b) Tiling 8×8 using 4×4.

Fig. 1: Different tiling schemes for an 8×8 multiplier.

Tiling can also be interpreted graphically as a *2-D* grid with one operand on the x-axis (MSB left, LSB right) and the other on the y-axis (MSB top, LSB bottom), and tiles are partial products. Each partial product's operands are derived by extracting the bits covered by its corresponding tile in both dimensions, while its shift size is determined by summing the LSB indices of tiles in both dimensions. Figure 1 demonstrates two tiling schemes for 8×8 multiplier through color-coded tiles, annotated shift operations (lime-green), and adders (cyan-blue). Sixteen 2×2 multipliers require more operations but offer flexibility as in Figure 1a. On the other hand, four 4×4 multipliers require only four additions and three shift operations. Figure 1b illustrates the Schoolbook decomposition showing the shift amount for each tile (representing sub-multipliers in decomposition terminology) and the addition operations required between sub-multipliers. The Karatsuba decomposition's computation reuse is visually represented in Figure 2 through a red line connecting sub-multipliers, denoting where a single multiplier can substitute for two while reusing existing computations [28]. Tiling strategies can be optimized for specific hardware architectures by incorporating architectural constraints into the decomposition process. For instance, AMD's UltraScale+ FPGA features 27×18-bit DSP multipliers; thus rectangular tiles matching these dimensions maximize DSP utilization, while significantly smaller tiles can be efficiently implemented using LUT-based logic.

C. DSP Block Consideration

1) Overview and Applications

A DSP block is a hard block within an FPGA that contains dedicated, efficient adders and multipliers that

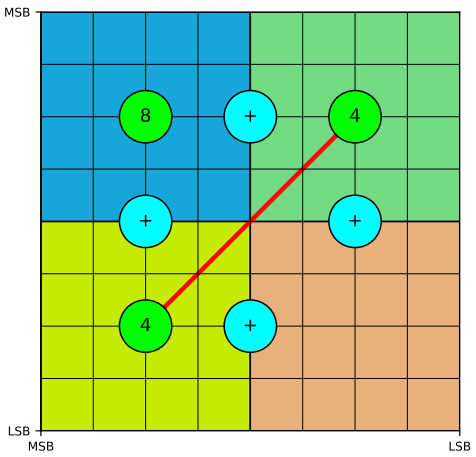


Fig. 2: Karatsuba decomposition of an 8×8 multiplier. The red line connects two multipliers that can be replaced by one.

run at high frequency. Its structure and capabilities have evolved over time. For instance, the Xilinx Virtex 5 architecture included a 25×18 multiplier and post-adder/accumulator [30]. The Xilinx Virtex 6 and 7 architectures introduced a 25-bit pre-adder while retaining the same 25×18 multiplier [31]. Later, the Ultrascale(+) architectures expanded the multiplier dimensions to 27×18 and increased the pre-adder wordlength to 27 bits [32]. All these architectures have the same 48-bit post-adder wordlength. The latest AMD Versal architecture improves these features by incorporating a 27×24 multiplier and larger 58-bit post-adder [33].

DSP blocks are versatile and multifunctional, supporting a wide range of applications. These include FIR filters [34], sensor processing [35], machine learning inference [36], [37], and radio astronomy [38]. Intel FPGA DSP blocks were expanded with some floating point capabilities [39] and the current AMD Versal DSP blocks can also implement a floating point operation per block in a distinct mode.

2) AMD DSP Slice Architecture

The internal structure of the AMD DSP block (named a Slice) along with its control signals, enables it to perform a wide variety of expressions. As shown in Figure 3, the DSP slice includes a pre-adder, a multiplier, multiplexers, and an ALU. It also features control signals to define its behavior. For instance, the OPMODE signal determines the configuration of the WXYZ multiplexers, while the ALUMODE signal specifies the expression computed by the ALU [40]. Additionally, the DSP block has pipeline registers that can be toggled on or off, allowing a tradeoff between latency and throughput. Enabling all four pipeline registers optimizes the multiply-accumulate operation to run at maximum frequency. DSP slice inputs support

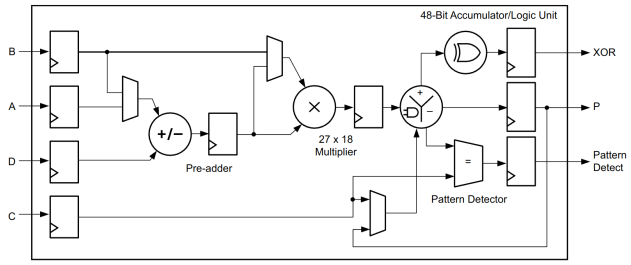


Fig. 3: Simplified DSP48E2 Slice architecture.

cascaded operation through ports such as PCIN and ACIN of another slice in the DSP column or independent processing via global ports such as A and B, offering flexible data routing options.

By manipulating control signals, any of the expressions presented in Equation 4 can be implemented using a single DSP slice:

$$DSPOut = \begin{cases} C \pm (B \times (D \pm A) \ + \ W + C_{in}) \\ C \pm (A \times (D \pm B) \ + \ W + C_{in}) \\ C \pm (B \times (D \pm B) \ + \ W + C_{in}) \\ C \pm (A \times (D \pm A) \ + \ W + C_{in}) \\ C \pm ((D \pm B)^2 \ + \ W + C_{in}) \\ C \pm ((D \pm A)^2 \ + \ W + C_{in}) \end{cases} \tag{4}$$

The cascade path enables chained product accumulation by routing PCOUT from a preceding DSP slice to PCIN of the current slice, feeding the Z multiplexer. Proper OPMODE and ALUMODE configuration select PCIN and define the arithmetic operation in Equation 5 that can be performed where W, X, Y, Z represent multiplexer outputs.

$$ALU \ \ Expression = \begin{cases} Z + W + X + Y + C_{in} \\ Z - (W + X + Y + C_{in}) \\ -Z + (W + X + Y + C_{in}) \\ \text{not}(Z + W + X + Y + C_{in}) \end{cases} \tag{5}$$

The DSP multiplier requires proper OPMODE configuration to route the partial products U and V through the X and Y multiplexers, respectively, while simultaneously selecting the appropriate input source (such as PCIN when implementing cascade chaining) for the Z multiplexer. This configuration enables both the fundamental multiplication operation and optional cascading functionality through careful signal routing control.

3) Cascading Two DSP Slices

Two DSP slices can be cascaded to create a 44×18-bit signed multiplier. Assuming that we have two inputs K

of size 44 bits and M of size 18 bits, then Equation 6 can be used.

$$\begin{aligned} K_L &= \{0, K[16:0]\} \\ K_H &= K[43:17] \\ K &= (K_L + K_H << 17) \\ K \times M &= (K_L \times M) + (K_H \times M << 17) \end{aligned} \tag{6}$$

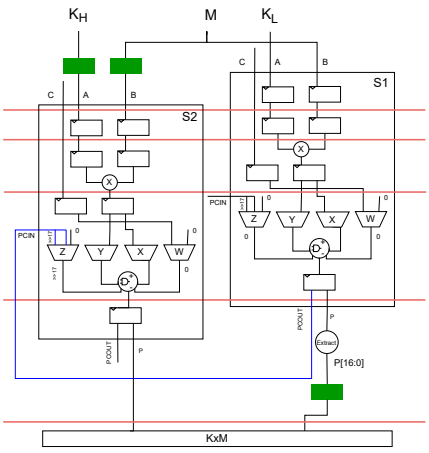
Then, due to the limited 48-bit size and to make use of the 17-bit right shift in the DSP slice, we can rewrite $K \times M$ using Equation 7.

$$\begin{aligned} P1 &= (K_L \times M) \\ P2 &= (K_H \times M) \\ K \times M &= \{(P2 + (P1 >> 17)),\ P1[16:0]\} \end{aligned} \tag{7}$$

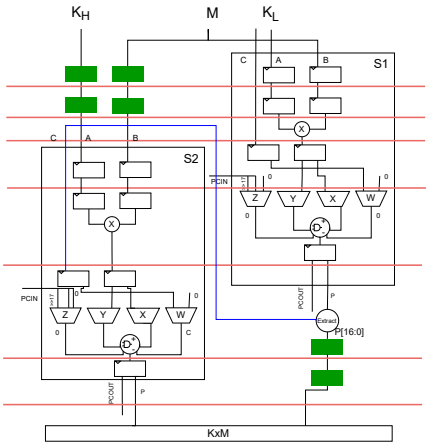
Figure 4 illustrates the cascaded DSP implementation with 4-cycle latency per slice when operating at maximum frequency. The design routes S1's P output (17 LSBs) to global routing while cascading PCOUT to S2's PCIN (blue line in Figure 4a), with S2's OPMODE configured for 17-bit PCIN shifting. Proper signal synchronization requires delaying K_H and M inputs by one clock cycle for S2 synchronization, and the same for P1's 17 LSBs before concatenation with P2 to form the final product, yielding a 5-cycle total latency. The cascade path only supports 0 or 17-bit shifts. For other shift values, a LUT-based shift and sign extension, and the global routing P to C path is used as in Figure 4b. Additional pipeline registers must be inserted to account for this additional cycle, resulting in a longer latency. Figure 4 shows the registers implemented in the logic fabric in green.

D. Signed vs Unsigned Multiplication

When decomposing large multipliers, careful consideration must be given to signed versus unsigned operations. Unsigned multipliers can be directly decomposed into smaller unsigned multipliers with full bit utilization. However, signed decomposition requires special handling; only the most significant portions of each input should use signed multiplication, while all other products must be computed as unsigned operations. This constraint becomes particularly important when using signed base primitives, which require zero-padding for unsigned operations. For instance, a signed 64×64-bit base multiplier can only scale to a 127×127-bit multiplier, as the least significant portions need MSB zero-padding. The general formula $(base\_size - 1) * 2^n + 1$ governs this relationship, producing valid sizes of 127, 253, 505, and 1009 bits for decomposition levels n=1 through 4 when starting from a 64×64 base.



(a) Cascade Path



(b) P to C path

Fig. 4: 44×18 Multiplier using two cascaded DSP blocks with different paths. The red lines represent the synchronization barriers. Green rectangles represent pipeline stages added outside of slices. The blue line distinguishes between the cascade path and the P-to-C path.

E. IP Multipliers

AMD provides a Vivado IP block [41] that can be used to build parametrized multipliers up to 64×64 bits, with options to optimize for frequency or resource utilization, reporting the resulting number of pipeline stages. At sizes larger than 47×47 bits, only the performance-optimal design is offered. While this achieves maximum DSP block frequency, it uses a large number of DSP blocks.

III. Large Multiplier Design

Large integer multiplication has been implemented using the different decompositions discussed in Section II. Some work has explored manual decomposition for not-so-small operand sizes, as in [25], [26], [28]. The authors of [20] used an automatic equality saturation framework

along with a cost model to hierarchically construct large multipliers from smaller ones while deciding which decomposition scheme should be used at each level. However, their approach does not consider the underlying architecture of the FPGA, relying on an HLS base multiplier, and thus does not achieve high frequency or area efficiency.

A. Hierarchical Multiplication

We employ a hierarchical construction approach for large integer multipliers, recursively decomposing the multiplication operation through multiple levels while flexibly applying different decomposition schemes, Schoolbook or Karatsuba, at each stage. Additionally, an architecture-optimized multiplier that achieves both high frequency and resource efficiency is used as a base multiplier.

1) Building a Base Multiplier

The selection of base multiplier size introduces a critical design trade-off: too small and there is increased LUT overhead for result combination, while too large and it constrains the feasible sizes for generated multipliers. Our baseline is the AMD Vivado IP block [41], which offers a maximum size of 64×64 bits. We also build our own base multiplier, using the tiling algorithm in [26], while considering the base DSP block architecture and exploiting the cascade paths where possible to run at maximum frequency. This hand-crafted base multiplier better exploits the capabilities of the DSP blocks to be more area efficient. It utilizes fewer overall resources while being larger, 78×78 bits, utilizing 14 DSP slices, tiled as illustrated in Figure 5. The x-axis shows one input (MSB left), while the y-axis shows the other (MSB top). Most tiled multipliers are 26×17 to fit the unsigned range of the DSP48E2, except those containing either operand's MSB, in which case leading zeros are not required as explained in Section II-D. Therefore, the full size at one or both of the 27×18 multiplier operands can be used, i.e. the 27-bit operand as in M10, the 18-bit operand as in M4, or at both as in M7.

The implementation process begins by efficiently mapping the generated multiplication terms (tiles) to DSP slices through several key optimizations. First, terms are sorted in ascending order based on their shift size, enabling systematic bit extraction at each stage while propagating the remaining bits to the next slice. This process inherently converts left shifts into right shifts using Equation 6 and 7. All operations are carefully constrained within the 48-bit output limit, preventing unnecessary LUT or DSP overhead to maintain optimal DSP block utilization. Non-overlapping least significant bits from each multiplier are directly routed to their corresponding positions in the final product, ensuring that additions with sign extension never exceed the 48-bit threshold. With naive ordering, overlapping bits could exceed this limit and thus require wider additions and wasteful resource usage. The required right shift size matches the number of non-overlapping bits between consecutive DSP slices. The implementation uses two distinct data paths selected by shift requirements: the dedicated cascade path handles 17-bit or zero shifts without additional logic, while non-standard shifts utilize the P-to-C path. In the latter case, the shift amount determines both the LSBs routed directly to the final product and the remaining bits' transformation (right-shifted and sign-extended to 48 bits for the next DSP's C input). The design primarily uses P-to-C paths with two exceptions: M1 to M12 and M6 to M7 connections utilize

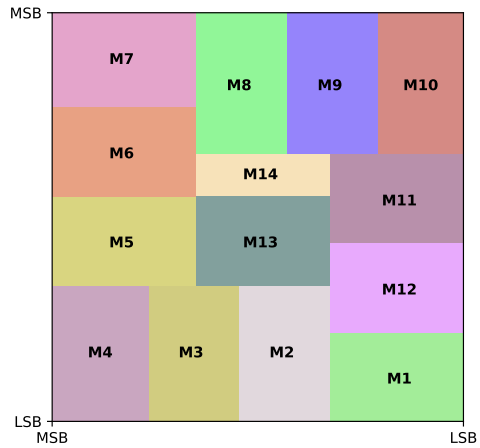


Fig. 5: Tiling scheme for 78×78 primitive base multiplier.

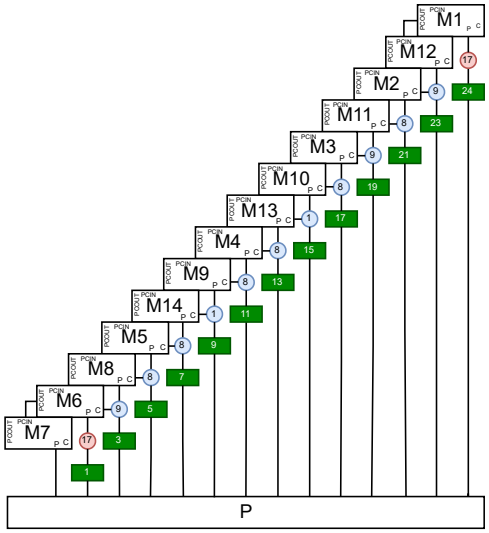


Fig. 6: Design of 78×78-bit primitive base multiplier. Green rectangles denote the number of pipeline stages. Red circles denote extraction operations while blue circles denote extraction, right shift, and sign extension operations.

TABLE I: AMD IP Multiplier vs Proposed Primitive Multiplier

	Size	LUTs	FFs	DSPs	Latency (Cycles)	#Wasted DSPs
IP	64	219	722	16	18	6.73
Primitive	78	537	1051	14	28	0.235

cascade paths. Section II-C3 details the synchronization rules governing cascade path operation, with Figure 6 providing a visual representation. Green rectangles show pipeline delays for timing synchronization, red circles indicate bit extraction for the final product, and blue circles represent extraction with right-shift and 48-bit sign-extension for the next slice's input. Numbers inside circles specify bit counts or shift amounts, optimizing data flow efficiently.

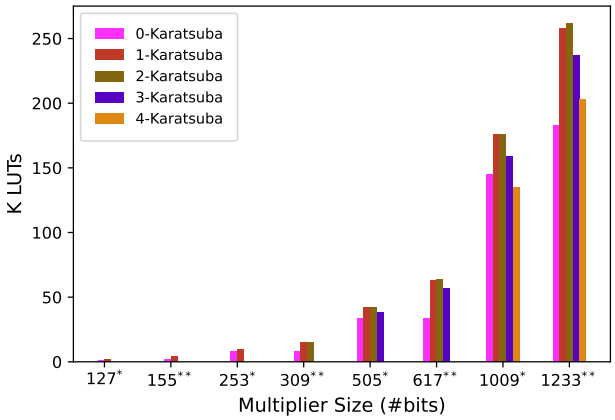
The AMD IP design methodology presented in [41] is followed to generate a 64×64-bit IP base multiplier. Our optimized base multiplier matches the AMD IP's maximum frequency while supporting larger inputs, and using 2 fewer DSPs and 300 more LUTs. For optimal resource allocation, we calculate the minimum number of 26×17 tiles needed to cover the 64×64 IP, or 78×78 primitive-based multipliers (Equation 8), i.e. excluding the forced waste due to unsigned tile use. While the IP multiplier wastes around 7 DSP slices beyond the theoretical minimum, our primitive-based design only sacrifices a few bits in one slice.

$$\text{optimal\_num\_DSPs} = \frac{(Size \times Size)}{(26 \times 17)} \tag{8}$$

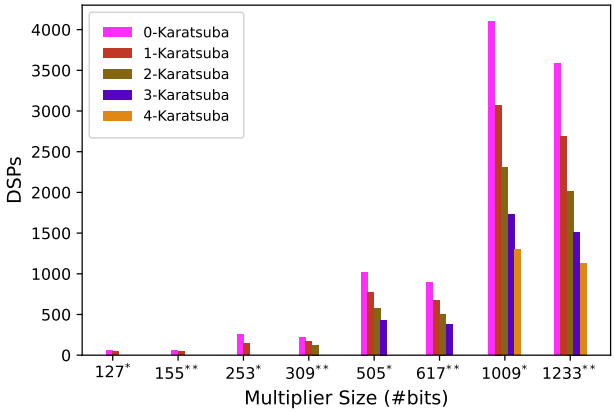
2) Multiplier Generation

We developed a tool that automatically generates large integer multipliers through combining smaller multipliers. Using the mathematical foundations described in Section II, the tool can take a base multiplier design and tile it an integer number of times in each dimension. The implementation supports mixed decomposition strategies, allowing designers to specify combinations of Schoolbook and Karatsuba methods through a JSON configuration file. The tool's output produced for each generated multiplier includes:

1) Complete Verilog implementations for each multiplier in the decomposition hierarchy and the combinational logic associated with it, along with corresponding testbenches for verification.
2) Tcl scripts to create Vivado projects, analyze resource utilization, and determine maximum achievable frequency while meeting timing constraints.
3) Dataflow decomposition diagrams that visually represent the mathematical operations and pipeline synchronization points.



(a) LUT Utilization with Different Decomposition Schemes



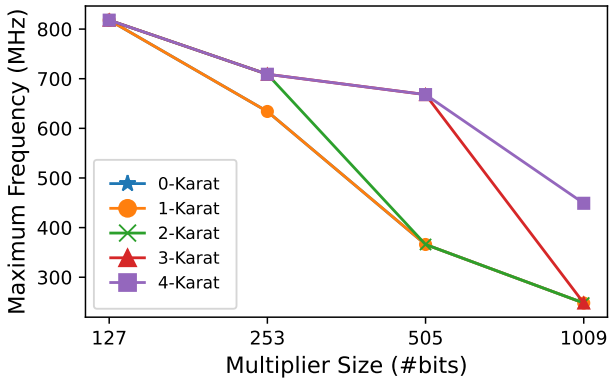
(b) DSP Utilization with Different Decomposition Schemes

Fig. 7: Resource Utilization for the IP-based (*) and Primitive-based (**) multipliers, with varying numbers of Karatsuba levels.

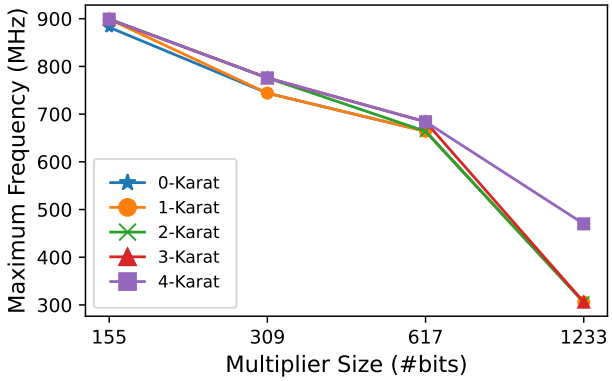
IV. Evaluation

A. Experiments and Results

We target the AMD Alveo U250 FPGA (XCU250) for fair comparison with prior work. We tiled both our 64×64 IP base multiplier and our own 78×78 base multiplier hierarchically, varying the number of Karatsuba and Schoolbook decompositions levels as in [20]. Our analysis compares these implementations against previous work across three key metrics: frequency, LUT utilization, and DSP utilization. Due to signed multiplication constraints, we built a 1009-bit multiplier with the IP base and a 1233-bit multiplier with our proposed base, then normalized the results to 1024-bit equivalents for comparison with previous research. Figure 7 shows how resource utilization scales with multiplier size and Karatsuba decomposition levels. More Karatsuba levels reduce DSP usage but increase LUT



(a) IP-based Multiplier Frequency Drop



(b) Primitive-based Frequency Drop

Fig. 8: Frequency Drop in Decomposition Levels in the Primitive-based and the IP-based multiplier

overhead due to added composition complexity. The quadratic growth in resource usage with input size led us to employ a quadratic scaling model for accurate 1024-bit comparisons.

Figure 8 depicts how the maximum frequency drops when the multiplier size increases. There is a large drop once the design spans more than one SLR, due to the reduced connectivity between SLRs. Figure 8a shows that the IP-based design suffers from this drop at lower sizes as it uses more DSP blocks. Meanwhile, the primitive based design, in Figure 8b, sustains higher frequency for larger sizes due to its better DSP block efficiency. Recall that Karatsuba decomposition reduces DSP block usage, so helps in both designs.

Table II compares our IP-based and primitive-based large multipliers against an HLS-based design and those generated by IMpress in terms of resource utilization and performance. Our IP-based multipliers consistently achieved significantly higher frequencies than both IMpress and HLS implementations across all decomposition schemes while being less efficient in terms of resource

TABLE II: Area and performance comparison between all designs. Our IP-based and Primitive-based multipliers achieved much higher frequency than other approaches, with the Primitive being more resource-efficient. * This column calculates the wasted DSPs by comparing the actual implementation against a scaled theoretical optimum.

	#Karat. Levels	LUTs	FFs	DSPs	Freq (MHz)	Latency (Cycles)	#Wasted DSPs *
HLS	3	71893	N/A	1476	256	17	475
	4	88986	N/A	1296	305	21	545
	5	126193	N/A	972	302	21	409
	6	165618	N/A	729	298	24	307
IMpress1	N/A	150400	N/A	810	250	23	247
IMpress2 [20]	N/A	129338	N/A	945	295	22	382
Primitive	4	143937	183155	837	528	229	87
	3	218891	299941	1131	453	214	130
	2	288082	381662	1550	438	193	216
	1	50936	68165	1709	442	167	70
	0	119262	139371	2471	428	136	99
IP	4	138860	66408	1328	445	220	578
	3	164508	27374	1760	440	211	759
	2	201416	91435	2432	300	196	1097
	1	173580	872846	3000	246	176	1221
	0	149237	247251	4219	253	125	1846

utilization because the 64×64 is not so efficient in the first place. Our primitive-based large multipliers achieved higher frequency than the IP-based multipliers in all different configurations in addition to being resource efficient, utilizing fewer DSPs, but slightly more LUTs. IMpress achieves better overall performance than the HLS design in terms of frequency and area utilization. Our primitive-based multiplier achieved at least 45%, and up to 73% better frequency than the best-performing multiplier in the HLS and IMpress implementations.

Figure 9 compares the best-performing models in different implementations in terms of the maximum achieved frequency against their LUTs and DSP usage. The results indicate distinct performance-resource tradeoffs: IMpress2 has the least LUT utilization, but has the second-highest DSP utilization while not achieving high-frequency. On the other hand, the full Karatsuba HLS implementation uses the least number of DSPs, but uses the largest number of LUTs and achieves the same low frequency as IMpress. Our IP-based approach achieved a much higher frequency than other implementations using LUTs efficiently, but also using more DSP blocks than other implementations. Our primitive-based multiplier with 4 levels of Karatsuba decomposition (the grey circle) achieved by far the highest frequency while using an LUTs and DSPs within the range of other implementations. Though they do achieve higher frequency, and hence throughput, our implementations do suffer from longer latency due to the deep pipelining required to map efficiently to the DSP blocks.

The DSP efficiency of each implementation is evaluated by comparing its utilization against a scaled theoretical optimum calculated from Equation 8. The base square size is 16 bits for HLS, 32 for IMpress, and 64 for our implementation. This optimal reference is

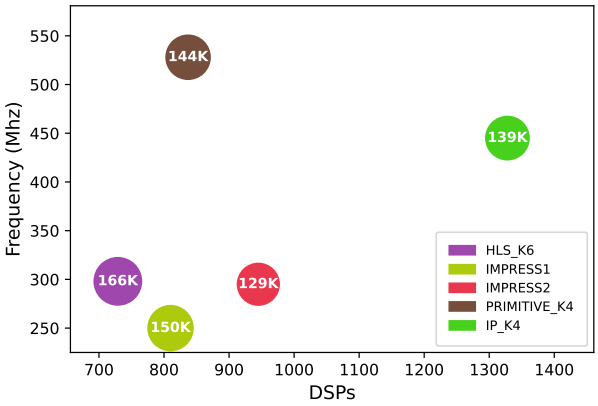


Fig. 9: Area and Performance comparison of different implementations. The size of each circle and the number inside denotes LUT utilization.

calculated by scaling the baseline DSP counts, where each Karatsuba decomposition level introduces 3 multipliers and each Schoolbook level 4 multipliers, that is ($base\_opt\_DSP\_count * 3^{\#Karatsuba} * 4^{\#Schoolbook}$). IP-based implementations achieve the poorest resource efficiency with the highest DSP waste, primarily due to the inherent inefficiency of the 64×64 IP base multiplier. In contrast, our primitive-based approach achieves maximal efficiency with minimal DSP waste. Intermediate implementations like HLS and IMpress show significantly higher DSP waste compared to our primitive-based design, though still outperform the IP-based approach. This shows that coupling the tiling automation with an efficient hand-crafted base primitive is key to maximising throughput and resource efficiency.

V. Related Work

Authors in [42] developed an automated tool for mapping arithmetic expressions to DSP blocks while preserving high operating frequency. They extended this work in [43] through multipumping techniques to enable efficient resource sharing. While these methods demonstrate effective DSP utilization, both approaches rely on fixed template databases that constrain their generalizability. Additionally, neither solution addresses the challenges of large integer multiplication. Authors in [20] presented IMpress, a framework that utilizes rewrite rules, E-graphs, and equality saturation to decompose a large multiplier into a hierarchy of smaller ones implemented using HLS. However, the base HLS multipliers do not achieve high frequency. Extending general decomposition schemes to be architecture-aware has been explored in previous work. In [26], the authors proposed a tiling algorithm to create a 58×58-bit multiplier that was previously obtained in [25] and to generalize for arbitrary wordlength multiplier and for arbitrary DSP multiplier width. The algorithm represents the input width, or a close number, as a linear combination of DSP multiplier input sizes. Then, the algorithm places rectangles on the boundary, then recursively solves the reduced square problem in the middle. However, it is limited to Schoolbook decomposition and square multiplier inputs, and it does not scale well for large numbers, because the reduced problem is not always a square area.

In [28], the authors implemented a rectangular multiplier with Karatsuba decomposition. They replaced subtractions in Equation 3 with additions to effectively use the post-adder of the DSP slice. However, this work lacks a scalable, generalizable algorithm for bigger operand sizes. Authors in [27] proposed a binary ILP formulation for integer multiplication, but it takes several days to compute sizes beyond 64×64 bits. Then, in [29], they extended their previous ILP approach to adopt Karatsuba decomposition as a special tile in the ILP problem along with some greedy and beam search heuristics. Both run at a low frequency and have a limited max operand size of 64 bits.

VI. Conclusions and Future Work

Since large integer multiplication is needed by many applications, it must be implemented in a way that maximizes performance while also using resources efficiently. In this work, we proposed an IP-based base multiplier as well as a new hand-crafted design of a base multiplier that can be used to form large integer multipliers, optimized for the AMD DSP48E2 primitive. We created a tool that automatically generates the RTL code for performing different decomposition schemes of large integer multiplication down to these two base multipliers. Both designs achieve high frequencies above 400MHz. The proposed Primitive-based approach achieved around 73% higher frequency than other approaches while using around 13% fewer DSP blocks at the expense of 5% more LUTs.

In this work, we hand-crafted the base multiplier around the DSP48E2 primitive. In the future, we plan to leverage equality saturation and rewrite rules to provide a tool that automatically builds the large integer multiplication, whether square or rectangle, directly from arbitrary DSP primitives. It would also be possible to extend this to other functions beyond multiplication. Finally, some strategies for mitigating the cross-SLR overhead, including another level of hierarchical pipelining could be explored.

VII. References

[1] S. Memik, A. Katsaggelos, and M. Sarrafzadeh, "Analysis and fpga implementation of image restoration under resource constraints," *IEEE Transactions on Computers*, pp. 390–399, 2003.

[2] P. Zemcik, "Hardware acceleration of graphics and imaging algorithms using fpgas," in *Proceedings of*

Spring Conference on Computer Graphics, 2002, pp. 25–32.

[3] A. G. Ye and D. M. Lewis, "Procedural texture mapping on fpgas," in *Proceedings of ACM/SIGDA seventh international symposium on Field programmable gate arrays*, 1999, pp. 112–120.

[4] J. Isoaho, J. Pasanen, O. Vainio, and H. Tenhunen, "Dsp system integration and prototyping with fpgas," *J. VLSI Signal Process. Syst.*, pp. 155–172, 1993.

[5] S. Demidenko, D.Johnson, K.Gribbon, and D. Bailey, *Implementing Signal Processing Algorithms in FPGAs: Digital Spectral Warping*, 2002.

[6] R. Tessier and W. Burleson, "Reconfigurable computing for digital signal processing: A survey," *Journal of VLSI signal processing systems for signal, image and video technology*, pp. 7–27, 2001.

[7] G. C. Chow, K. Eguro, W. Luk, and P. Leong, "A karatsuba-based montgomery multiplier," in *2010 International Conference on Field Programmable Logic and Applications*, 2010, pp. 434–437.

[8] D. D. Chen, G. X. Yao, R. C. Cheung, D. Pao, and Ç. K. Koç, "Parameter space for the architecture of fft-based montgomery modular multiplication," *IEEE Transactions on Computers*, vol. 65, no. 1, pp. 147–160, 2016.

[9] M. M. Islam, M. S. Hossain, MD. Shahjalal, M. K. Hasan, and Y. M. Jang, "Area-time efficient hardware implementation of modular multiplication for elliptic curve cryptography," *IEEE Access*, vol. 8, pp. 73 898–73 906, 2020.

[10] M. A. Mehrabi, C. Doche, and A. Jolfaei, "Elliptic curve cryptography point multiplication core for hardware security module," *IEEE Transactions on Computers*, vol. 69, no. 11, pp. 1707–1718, 2020.

[11] T. Ye, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Performance modeling and fpga acceleration of homomorphic encrypted convolution," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, 2021, pp. 115–121.

[12] A. C. Mert, E. Öztürk, and E. Savaş, "Design and implementation of encryption/decryption architectures for bfv homomorphic encryption scheme," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 353–362, 2020.

[13] P. G. Comba, "Exponentiation cryptosystems on the IBM PC," *IBM Systems Journal*, pp. 526–538, 1990.

[14] A. Karatsuba and Y. Ofman, "Multiplication of multidigit numbers on automata," *Soviet Physics Doklady*, vol. 7, p. 595, 1962.

[15] C. Yap and C. Li, "Quickmul: Practical fft-based integer multiplication," 2000.

[16] A. Emerencia, "Multiplying huge integers using fourier transforms," 2007.

[17] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, pp. 297–301, 1965.

[18] A. Schönhage and V. Strassen, "Schnelle multiplikation großer zahlen," *Computing*, pp. 281–292, 1971.

[19] N. Emmart and C. C. Weems, "High precision integer multiplication with a gpu using strassen's algorithm with multiple fft sizes," *Parallel Processing Letters*, pp. 359–375, 2011.

[20] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "IMpress: Large integer multiplication expression rewriting for FPGA HLS," in *Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–10.

[21] Beaumont, Boudet, Rastello, and Robert, "Partitioning a square into rectangles: NP-completeness and approximation algorithms," *Algorithmica*, p. 217–239, 2002.

[22] L. A. Levin, "Problems, complete in "average" instance," in *ACM Symposium on Theory of Computing*, 1984.

[23] ——, "Average case complete problems," *SIAM Journal on Computing*, pp. 285–286, 1986.

[24] M. Kumm, J. Kappauf, M. Istoan, and P. Zipf, "Resource optimal design of large multipliers for fpgas," in *IEEE Symposium on Computer Arithmetic (ARITH)*, 2017, pp. 131–138.

[25] F. Dinechin and B. Pasca, "Large multipliers with less DSP blocks," in *Field Programmable Logic and Applications (FPL)*, 2009.

[26] D. B. Roy, D. Mukhopadhyay, M. Izumi, and J. Takahashi, "Tile before multiplication: An efficient strategy to optimize DSP multiplier for accelerating prime field ecc for NIST curves," in *Design Automation Conference (DAC)*, 2014, pp. 1–6.

[27] M. Kumm, J. Kappauf, M. Istoan, and P. Zipf, "Resource optimal design of large multipliers for FPGAs," in *IEEE Computer Arithmetic (ARITH)*, 2017, pp. 131–138.

[28] M. Kumm, O. Gustafsson, F. De Dinechin, J. Kappauf, and P. Zipf, "Karatsuba with rectangular multipliers for FPGAs," in *Computer Arithmetic (ARITH)*, 2018, pp. 13–20.

[29] A. Böttcher, K. Kullmann, and M. Kumm, "Heuristics for the design of large multipliers for FPGAs," in *IEEE Computer Arithmetic (ARITH)*, 2020, pp. 17–24.

[30] Xilinx, Inc., "Virtex-5 FPGA user guide (UG190)," 2009.

[31] ——, "Virtex-6 family overview (DS150)," p. 1, 2015.

[32] ——, "Ultrascale architecture and product data sheet: Overview (DS890)," p. 44, 2024.

[33] ——. (2022) Versal ACAP DSP engine architecture manual (AM004).
[34] ——, "DSP: Designing for optimal results. high-performance DSP using virtex-4 FPGAs (UG073)," 2005.
[35] T. Hussain, S. Amin, U. Zabit, F. Kamran, O. D. Bernal, and T. Bosch, "A high performance real-time FPGA-Based interferometry sensor architecture," in *International Conference on Frontiers of Information Technology (FIT)*, 2016, pp. 130–135.
[36] D. Wang, K. Xu, J. Guo, and S. Ghiasi, "DSP-Efficient hardware acceleration of convolutional neural network inference on FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (IEEE TCAD)*, pp. 4867–4880, 2020.
[37] L. Ioannou and S. A. Fahmy, "Streaming overlay architecture for lightweight LSTM computation on FPGA SoCs," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 1, pp. 8:1–8:26, 2022.
[38] P. H. W. Leong, "Recent trends in FPGA architectures and applications," in *IEEE International Symposium on Electronic Design, Test and Applications*, 2008, pp. 137–141.
[39] M. Langhammer and B. Pasca, "Floating-point DSP block architecture for FPGAs," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 117–125.
[40] Xilinx, Inc., "Ultrascale architecture DSP slice user guide (UG579)," 2021.
[41] ——, "Multiplier v12.0 LogiCORE IP product guide (PG108)," 2015.
[42] B. Ronak and S. A. Fahmy, "Mapping for maximum performance on FPGA DSP blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 573–585, 2016.
[43] ——, "Multipumping flexible DSP blocks for resource reduction on xilinx FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1471–1482, 2017.